UQAC
Département d'informatique
et de mathématique
Université du Québec à Chicoutimi

8INF878 – Intelligence artificielle agentique : Travail No. 1

AUSSI V2 – Résolution de problème et généralisation

Etudiants :
CAPELO Thomas
LOUET Noah
..
..

16 octobre 2026

Table des matières

1. Introduction 2
2. Architecture générale ? 2
3. Labyrinthe 2
   3.1. Modélisation 2
   3.2. Heuristique 2
   3.3. Comparaison des algorithmes 2
4. N-Puzzle 2
   4.1. Modélisation 2
   4.2. Heuristique 2
   4.3. Comparaison des algorithmes 2
5. Jeu arithmétique 2
   5.1. Modélisation 2
   5.2. Heuristique 2
   5.3. Comparaison des algorithmes 3
6. Navigation 4
   6.1. Modélisation 4
   6.2. Heuristique 4
   6.3. Comparaison des algorithmes 4
7. Conclusion 4

1. Introduction

2. Architecture générale ?

3. Labyrinthe

3.1. Modélisation

3.2. Heuristique

3.3. Comparaison des algorithmes

Instance 1

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree						
BFS	Graph						
UCS	Tree						
UCS	Graph						
Greedy	Tree						
Greedy	Graph						
A*	Tree						
A*	Graph						

4. N-Puzzle

4.1. Modélisation

4.2. Heuristique

4.3. Comparaison des algorithmes

5. Jeu arithmétique

5.1. Modélisation

Le problème consiste à construire une expression permettant d'atteindre une valeur cible à partir d'un ensemble de nombres et d'opérateurs autorisés.

Un état est représenté par un ensemble de couples (valeur, expression) contenant chaque résultat intermédiaire ainsi que la formule ayant permis de l'obtenir. À chaque étape, deux valeurs sont sélectionnées puis combinées à l'aide d'un opérateur disponible parmi +, -, * ou /.

L'application d'une action remplace les deux opérandes par leur résultat, réduisant ainsi le nombre de valeurs présentes dans l'état. Un état but est atteint lorsqu'il ne reste plus qu'une seule valeur et que celle-ci est égale à la cible recherchée.

Chaque action possède un coût uniforme de 1.

5.2. Heuristique

On peut représenter l'Heuristique utilisée sous la forme :

$$h(s) = (|s| - 1) + \frac{d}{d+1}$$

où :

- $|s| - 1$ représente le nombre minimal d'opérations restantes
- d est l'écart minimal entre une valeur présente dans l'état et la valeur cible

Le premier terme privilégie les états proches d'une expression complète tandis que le second favorise les états contenant une valeur proche de l'objectif.

Cette heuristique n'est pas admissible. Elle a toutefois été retenue afin d'orienter la recherche vers les états les plus prometteurs et de comparer le comportement des algorithmes informés (Greedy et A*) avec celui des algorithmes non informés (BFS et UCS).

5.3. Comparaison des algorithmes

Instance Facile

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	3	3	1459	595	1146	0.053s
BFS	Graph	3	3	1459	595	1146	0.069s
UCS	Tree	3	3	1459	595	1146	0.054s
UCS	Graph	3	3	1459	595	1146	0.055s
Greedy	Tree	3	3	53	18	38	0.002s
Greedy	Graph	3	3	53	18	38	0.002s
A*	Tree	3	3	53	6	47	0.002s
A*	Graph	3	3	53	6	47	0.002s

Instance Moyenne

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	4	4	58473	16032	45964	2.467s
BFS	Graph	4	4	48897	14112	38308	2.207s
UCS	Tree	4	4	58473	16032	45964	2.397s
UCS	Graph	4	4	48897	14112	38308	2.202s
Greedy	Tree	4	4	579	515	77	0.024s
Greedy	Graph	4	4	579	515	77	0.026s
A*	Tree	4	4	731	187	544	0.029s
A*	Graph	4	4	731	187	544	0.029s

Instance Difficile

Algorithme	Mode	Coût total	Profondeur	Nœuds générés	Nœuds expansés	Max OPEN	Temps (ms)
BFS	Tree	N/A	N/A	265580	19612	245968	10s
BFS	Graph	N/A	N/A	217432	19312	198120	10s
UCS	Tree	N/A	N/A	255654	18783	236871	10s

UCS	Graph	N/A	N/A	194410	17034	177376	10s
Greedy	Tree	5	5	144	10	136	0.005s
Greedy	Graph	5	5	144	10	136	0.004s
A*	Tree	5	5	458	63	395	0.018s
A*	Graph	5	5	458	63	395	0.020s

Les résultats montrent clairement que l'heuristique joue un rôle important dans le jeu arithmétique. Les algorithmes non informés, BFS et UCS, voient leur nombre de nœuds générés augmenter très vite quand la taille du problème grandit. Sur l'instance difficile, aucun des deux ne trouve de solution avant d'atteindre la limite de temps fixée.

À l'inverse, Greedy et A* résolvent toutes les instances testées. Ils explorent aussi beaucoup moins de nœuds, parfois des milliers ou des millions de fois moins. Cette différence vient de l'utilisation de l'heuristique. Elle guide la recherche vers les états les plus prometteurs au lieu d'explorer l'espace d'états de façon uniforme. Bien que cette heuristique ne soit pas admissible, elle donne assez d'informations pour guider la recherche vers une solution. Elle ne garantit pas pour autant que la solution soit optimale.

On remarque aussi que BFS et UCS donnent les mêmes résultats. Cette ressemblance était attendue puisque toutes les actions ont un coût unitaire. Le coût cumulé utilisé par UCS est donc égal à la profondeur utilisée par BFS. Enfin, le mode Graph Search apporte un gain important pour les algorithmes non informés. Il évite d'explorer plusieurs fois les mêmes états, ce qui réduit le nombre de nœuds générés et la taille maximale de la frontière OPEN.

6. Navigation

6.1. Modélisation

6.2. Heuristique

6.3. Comparaison des algorithmes

7. Conclusion